

FORMAL VERIFICATION OF STATE EQUIVALENCE

Phase 3 - Lean 4 Proof Report

Core result

If two initial states agree on every operationally relevant variable of a statement S , then a terminating execution of S can be reproduced from the second state, and the two final states agree on those same variables.

Authors: Ehsan Soveizi & Mahdi Rashidi
Course: Theory of Languages and Machines
Department of Computer Engineering
Ferdowsi University of Mashhad

Selected theorem: Option 3 - State Equivalence

1. Instructor Correction and Motivation

Why the original conclusion must be weakened

Correction

The original project statement concluded equality of the two complete final states. This is not valid in general. The corrected theorem concludes agreement only on the free variables of S .

Why complete state equality is false

A statement can only observe or modify variables that occur in its syntax. If the initial states disagree on an unrelated variable, the execution has no reason to remove that disagreement.

Counterexample

Let S be $x := 1$. Assume s_1 and s_2 agree on x but disagree on z . Both executions assign 1 to x , while z is never read or written. Therefore, the final states still disagree on z . The complete states are not equal, although the executions are equivalent from the viewpoint of S .

Correct semantic interpretation

- The statement must make the same control-flow decisions in both executions.
- Every expression evaluated by the statement must have the same value in both executions.
- The final states must agree only on variables relevant to the statement.
- Differences on unrelated variables are allowed to remain.

This correction is not merely a technical weakening. It identifies the strongest generally valid conclusion that can be justified from agreement on the variables used by S .

2. Why This Hash Subset Was Chosen

Minimal syntax, complete proof coverage

The formalization intentionally uses a compact subset of Hash. The objective is not to model every surface feature of the language, but to include every semantic construct that introduces a distinct proof obligation.

Construct	Why it is included	Proof idea exposed
num, var	Base expression forms	Constants ignore the state; variables use state agreement.
add, mul, lt, eq	Representative binary expressions	Agreement must be restricted to both operands.
assignment	State modification	Updated variable versus unchanged variables.
sequence	Composition	Agreement must survive an intermediate state.
if-else	Control-flow choice	Equal guard values force the same branch.
while	Recursive execution	Induction over a finite terminating derivation.

Why other constructs are omitted

- A for-loop can be translated into initialization followed by a while-loop whose body ends with the update.
- break and continue require an explicit control-result type; they introduce engineering complexity outside the core theorem.
- Exceptions require abnormal semantic outcomes rather than an ordinary final state.
- Declarations, scopes, classes, and methods require binding and environment machinery that is unnecessary for this proof.

Design criterion

Each included construct contributes a genuinely different induction case. Each omitted construct would enlarge the semantics without changing the central State Equivalence argument.

3. Semantic Model

The objects and relations used by the proof

States and expressions

```
abbrev Var := String
abbrev State := Var -> Int
```

```
def evalExpr : Expr -> State -> Int
```

A state is a total function from variable names to integers. Expression evaluation is a total recursive function: for every expression and every state, Lean computes exactly one integer result.

State update

```
update s x v = fun y => if y = x then v else s y
```

The update operation changes exactly one variable. This definition directly determines the two branches of the assignment proof: the queried variable is either the assignment target or it is not.

Big-step statement semantics

BigStep S s t means that S has a terminating execution from state s to state t. Each constructor corresponds to one operational rule: assignment, sequence, true/false conditionals, and true/false loop rules.

- The semantics is relational because a statement execution is represented by evidence of a complete derivation.
- A diverging loop has no BigStep derivation and therefore no final state in this model.
- Induction on BigStep follows the actual execution structure, especially for while-loops.

Functional/relational split

Expressions are total, pure computations, so a function is natural. Statements may contain loops and require termination evidence, so a relation is more appropriate.

4. The Corrected Theorem

Exact statement and logical meaning

```
AgreeOn (freeVarsStmt S) s1 s2 ->
BigStep S s1 t1 ->
exists t2, BigStep S s2 t2 /\
  AgreeOn (freeVarsStmt S) t1 t2
```

Meaning of the assumptions

1. The initial states s1 and s2 agree on every variable relevant to S.
2. There is a terminating execution of S from s1 to t1.

Meaning of the conclusion

1. A corresponding final state t2 exists for an execution starting from s2.
2. The two final states agree on every variable relevant to S.

Why the result is existential

The theorem starts with one concrete terminating derivation. The proof mirrors that derivation from the second state and constructs a matching final state. It does not assume termination from every possible state; termination is transferred from one state to another under the required agreement condition.

What the theorem deliberately does not claim

- It does not prove $t1 = t2$ as complete state functions.
- It does not prove termination when no reference BigStep derivation is available.
- It does not cover language constructs absent from the selected semantics.

Central invariant

At every corresponding point of the two executions, the states agree on all variables that the remaining proof may need.

5. Relevant Variables and Agreement

How the proof formalizes what a statement can observe or change

Expression variables

freeVarsExpr e x holds when evaluating e may read x. A number has no variables, a variable expression selects its own name, and every binary expression uses the union of the variables of its operands.

Statement variables

`freeVarsStmt` records the complete operational support of a statement: variables read by expressions or guards and variables written by assignments.

Why the assignment target is included

For $x := e$, the theorem must conclude that the final states agree on x . Even when x is not read by e , it is modified by the statement and must therefore belong to the relevant-variable predicate.

Agreement on a predicate

```
def AgreeOn (P : Var -> Prop) (s1 s2 : State) : Prop :=
  forall x, P x -> s1 x = s2 x
```

`AgreeOn` is pointwise equality restricted by a predicate. It avoids full function equality and avoids introducing finite-set infrastructure. The same definition can express agreement on expression variables, statement variables, or a larger contextual set used by the strengthened induction theorem.

Terminology note

In programming-language theory, an assignment target is often classified as a written variable rather than a free variable. In this proof, `freeVarsStmt` should be read as the read/write support of S . The name follows the project statement, while the definition is tailored to the corrected result.

6. Why evalExpr Is a Function

Replacing relational expression evaluation without weakening the proof

The selected expression language is total, deterministic, pure, and structurally recursive. No expression can diverge, modify the state, request input, or produce multiple possible results. Therefore, a recursive function is a faithful and simpler semantic model.

Aspect	Functional evalExpr	Relational EvalExpr
Result	Returns one integer directly	Requires a derivation <code>EvalExpr e s v</code>
Determinism	Built into the function	Must be proved as a separate property
Use in this theorem	Sufficient and concise	Correct but unnecessarily verbose
Induction still required?	Yes - induction on Expr	Yes - induction on the evaluation derivation

What was removed

Only the extra inductive derivation object for expressions was removed. The essential semantic argument remains: the proof must still show that two different states produce the same expression value when they agree on the variables read by that expression.

When a relation would become necessary

- Nondeterministic expressions or input operations.
- Expression-level exceptions or undefined results.
- Side effects during expression evaluation.
- Expression forms that may not terminate.

Conclusion

Using `evalExpr` is correct for this subset and matches the functional expression semantics presented in the project guide. It does not bypass the locality proof.

7. Local Lemma: Expression Locality

Step-by-step structural induction on expressions

```
eval_eq:
  AgreeOn (freeVarsExpr e) s1 s2 ->
    evalExpr e s1 = evalExpr e s2
```

This lemma is stronger than ordinary determinism. The states may differ globally; only their values at the variables used by e must be equal.

Induction cases

1. num n: evaluation returns n in both states, so the equality is reflexive.
2. var x: freeVarsExpr (var x) x is immediate; AgreeOn supplies $s1\ x = s2\ x$.
3. add and mul: restrict the parent agreement to the left and right operand, apply both induction hypotheses, and rewrite the two operand results.
4. lt and eq: first prove equal operand values, then the same comparison condition is evaluated on both sides, producing the same integer encoding of true or false.

The role of Or.inl and Or.inr

For a binary expression, the relevant-variable predicate is a disjunction. Evidence that x belongs to the left operand is injected with Or.inl; evidence for the right operand is injected with Or.inr. These constructors allow the parent agreement assumption to be specialized for each induction hypothesis.

Why this lemma is reused

Assignment expressions and all conditional or loop guards depend on exactly the same locality principle. Proving it once avoids repeating structural induction inside every BigStep case.

8. Why the Auxiliary Theorem Must Be Strengthened

The key proof-design decision

A direct induction that preserves agreement only on freeVarsStmt of the current substatement is too weak. The problem appears immediately in sequence and recursively in while-loops.

Sequence problem

During S1, the proof must preserve not only variables used by S1, but also variables that S2 will need later. An induction hypothesis limited to freeVarsStmt S1 cannot guarantee this.

Strengthened statement

```
aux:
  BigStep S s1 t1 ->
    (forall x, freeVarsStmt S x -> P x) ->
      AgreeOn P s1 s2 ->
        exists t2, BigStep S s2 t2 /\ AgreeOn P t1 t2
```

Meaning of P

P is an arbitrary ambient predicate that contains every variable of the current statement. The auxiliary theorem preserves agreement on all of P, including variables needed by later statements or later loop iterations.

Why this formulation is concise

An alternative formulation explicitly carries a context predicate R and repeatedly proves agreement on freeVarsStmt S union R. The current version absorbs both components into one predicate P and records only the inclusion freeVarsStmt S subset P. This avoids deeply nested disjunctions without changing the mathematics.

General proof technique

When an induction hypothesis is too weak for a compositional case, strengthen the proposition before performing induction. The stronger result is easier to use and the original theorem becomes a final specialization.

9. BigStep Induction - Assignment Case

How the proof reproduces a state-changing command

Suppose the reference derivation executes $x := e$ from s to update s x (evalExpr e s). The proof must construct the corresponding assignment execution from s2 and prove that the two updated states still agree on P.

1. Restrict hSub to the variables of e. Since every expression variable of e is also a variable of $x := e$, hSub shows that it belongs to P.
2. Use hAgree to obtain equality of the states on those expression variables.
3. Apply eval_eq and derive evalExpr e s = evalExpr e s2.
4. Construct the second derivation with BigStep.assign. Its final state is update s2 x (evalExpr e s2).
5. Fix an arbitrary y in P and split on $y = x$.

Case $y = x$

Both updated states return their newly assigned values. Those values are equal by `eval_eq`, so agreement holds at x .

Case $y \neq x$

Both updates return the old state values. Since y belongs to P , `hAgree` gives $s\ y = s2\ y$. Therefore agreement is preserved at every unchanged variable in P .

Lean mechanism

The `by_cases` split matches the definition of `update` exactly. `simp` unfolds the conditional update and closes each branch using either the expression equality or the original state agreement.

10. BigStep Induction - Sequence Case

Preserving future information through an intermediate state

A reference execution of $S1; S2$ contains an intermediate state `smid`. The second execution must construct a corresponding `smid2` and then continue with $S2$.

1. From `hSub` for the whole sequence, derive `hSub1`: every variable of $S1$ belongs to P .
2. Apply `ihFirst` to reproduce $S1$ from $s2$. Obtain `smid2`, a BigStep derivation for $S1$, and agreement on all of P between `smid` and `smid2`.
3. Derive `hSub2`: every variable of $S2$ also belongs to P .
4. Apply `ihSecond` from the two intermediate states. The required starting agreement is exactly `hMidAgree` from the first induction hypothesis.
5. Combine the two reproduced derivations with `BigStep.seq` and return the final agreement supplied by `ihSecond`.

Why the strengthened theorem is essential here

If `ihFirst` preserved only the variables of $S1$, `smid` and `smid2` could disagree on a variable used only by $S2$. The second application of the induction hypothesis would then be impossible. Preserving the ambient P prevents this loss of future information.

Compositional structure

The proof follows the semantic rule itself: reproduce the first derivation, transfer the preserved invariant to the intermediate states, then reproduce the second derivation.

11. BigStep Induction - Conditional Cases

Equal guard values force identical branch selection

True branch

1. Use `hSub` and `hAgree` to establish agreement on `freeVarsExpr b`.
2. Apply `eval_eq` to prove `evalExpr b s = evalExpr b s2`.
3. The reference guard is nonzero. If the second guard were zero, equality would make the reference guard zero, contradicting `hGuard`.
4. Restrict `hSub` to the variables of the then-branch and apply `ihThen`.
5. Construct the second `ifTrue` derivation using the transferred nonzero guard and the reproduced branch execution.

False branch

1. The same locality argument gives equality of the guard values.
2. The reference guard equals zero, so the second guard also equals zero by symmetry and transitivity.
3. Restrict `hSub` to the else-branch and apply `ihElse`.
4. Construct the second `ifFalse` derivation and reuse the final agreement produced by the induction hypothesis.

Key semantic consequence

The two states may differ outside P , but the guard cannot observe those differences. Both executions therefore select the same branch.

No determinism theorem for BigStep is required. The branch is synchronized directly through equality of the guard evaluations.

12. BigStep Induction - While Cases

The central recursive part of the proof

whileFalse

The reference guard evaluates to zero. Expression locality transfers the same zero result to s2, so the second loop also terminates immediately. Its final state is s2, and the original hAgree is already the required final agreement on P.

whileTrue

The reference derivation contains a nonzero guard, one body execution from s to smid, and a recursive execution of the remaining loop from smid to sout.

1. Transfer the nonzero guard result to s2 using eval_eq.
2. Restrict hSub to body variables and apply ihBody. This constructs smid2 and preserves agreement on all of P.
3. Apply ihRest to the recursive loop derivation, starting from smid and smid2. The starting agreement is hMidAgree.
4. Obtain t2 and the reproduced remainder of the loop together with final agreement on P.
5. Combine the transferred guard, reproduced body, and reproduced remainder using BigStep.whileTrue.

Why induction is on the derivation

A terminating loop derivation is finite and explicitly contains the derivation for the remaining iterations. Structural induction on this evidence provides exactly the recursive hypothesis required by ihRest.

This case also explains why P must include guard variables across iterations: after the body changes the state, the next guard evaluation must still be synchronized.

13. Final Specialization and Worked Example

How the generalized proof returns the required theorem

Final specialization

After aux is established, the main theorem requires no new semantic reasoning. Set P to freeVarsStmnt S. The inclusion premise becomes the identity implication, and the initial AgreeOn assumption is exactly the theorem assumption.

```
exact aux hExec (freeVarsStmnt S) s2
(fun _ hx => hx) hInitial
```

The short ending is evidence that the proposition was strengthened correctly: all difficult reasoning has already been handled in the reusable induction theorem.

Worked example

Statement

$S = x := y + 1$

Variable	s1	s2	Relevant to S?
x	0	42	Yes - assignment target
y	7	7	Yes - read by the expression
z	20	99	No

Both expressions evaluate to 8 because the states agree on y. Both executions assign 8 to x and preserve y = 7. Therefore, the final states agree on x and y. The unrelated values of z remain different, which confirms why complete state equality is neither expected nor required.

Proof flow in one sentence

Expression locality synchronizes computed values and guard decisions; the strengthened BigStep induction then preserves an ambient agreement predicate through every semantic rule.